Bases de données
SQL

Fonctions et procédures

SQL_06a
v122c

2025-02-11

Christina.Khnaisser@USherbrooke.ca
Luc.Lavoie@USherbrooke.ca

© 2018-2021, **Μῆτις** (http://info.usherbrooke.ca/llavoie)
CC BY-NC-SA 4.0 (https://creativecommons.org/licenses/by-nc-sa/4.0/)

Plan

- Motivation
- Un exemple
- Les fonctions
- Les procédures
- Les formes de l'abstraction
- Exemples
- Références

Fonctions et procédures
Motivation

- Qu'ont en commun une fonction, une procédure et un automatisme ?
  - Ce sont des abstractions paramétrables d'un traitement.
- Qui fait quoi ?
  - Une *fonction* **calcule une valeur** suite à un appel (sur la base de ses paramètres ou de l'état de la BD).
  - Une *procédure* **change l'état de la BD** suite à un appel (sur la base de ses paramètres **et** de l'état de la BD).
  - Un *automatisme* change l'état de la BD **suite à un évènement** (sur la base de ses paramètres et de l'état de la BD).

MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MiTiS — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Ne pas oublier que « ou » est inclusif, du moins en logique, en français, en allemand, en italien, en espagnol et en anglais.
Pour l'américain, on ne sait pas vraiment.

Il est fréqent de distinguer les fonctions selon qu'elles prennent en compte l'état ou non. Les premières sont qualifiées de muables ou stables (*mutable, stable*) et le secondes d'immuables (*immutable*, *immediate* ou *pure*).

Si on utilise les types abstraits algébriques (TAA), la fonction et le la procédure coïncide avec l'observateur et le transformateur

observateur
* aucun paramètre : constante
* in : liste de paramètres : fonction (immuable)
* in : liste de paramètres + état : fonction (stable)

transformateur - in : lp + état; out : lp + état
* sur appel : procédure
* sur changement d'état : contrainte
* sur événement : action compensatoire (automatisme, trigger)

Réflexion
Sauriez-vous expliquer pourquoi il n'y a pas d'automatismes «fonctionnels» (calculant une

valeur sans changer l’état) ?

Portée

- Le présent module traite des fonctions et procédures.
- Le module SQL_06b traite des automatismes.

MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MTIS — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Exemple

Exemple 0
Domaines SQL

Définition de domaines utilisés dans le cadre des prochains exemples.

Le numéro de téléphone ne comprend que les chiffres qui le composent (aucune marque d'édition telle que tirets, parenthèses, points, etc.).

Les numéros des pays membres de l'ITU comprennent entre 8 et 13 chiffres.

Les chiffres des numéros canadiens et états-uniens se répartissent ainsi :

- indicatif national : 1;
- indicatif régional : 3;
- équipement : 7.

```
CREATE DOMAIN Telephone
  VARCHAR(13)
  CONSTRAINT Telephone_ITU
    CHECK(VALUE SIMILAR TO '[0-9]{8,13}');

CREATE DOMAIN Telephone_CA
  Telephone
  CHECK(VALUE SIMILAR TO '[1][0-9]{10}');

CREATE DOMAIN Montant_CAD
  NUMERIC(12,2)
  CHECK(0 <= VALUE);
```


MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MVSS — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Exemple 0
Fonction SQL

Déterminer l'indicatif régional d'un numéro de téléphone canadien ou états-unien.

Version ISO

```
CREATE FUNCTION IndicatifRegional
   (t Telephone_CA)
   RETURNS VARCHAR(3)
RETURN
   SUBSTR(t, 2, 3) ;

...

SELECT nom, prenom, telephone, courriel
FROM Abonne
WHERE IndicatifRegional(telephone) = '819';
```


MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MTIS — CC BY-NC-SA 4.0

Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Exemple 0
Contexte SQL

Contexte du prochain exemple

```
CREATE DOMAIN NoProduit
  CHAR(8)
  CONSTRAINT NoProduit_co
    CHECK(VALUE SIMILAR TO '[0-9]{8}');

CREATE DOMAIN CodePromotionnel
  VARCHAR(15)
  CONSTRAINT CodePromotionnel_co
    CHECK(VALUE SIMILAR TO '[A-Za-z0-9]+');

CREATE TABLE Produit
  (
    noProduit NoProduit,
    nom Text,
    prix Montant_CAD,
    CONSTRAINT Produit_cc0 PRIMARY KEY (noProduit)
  );

CREATE TABLE Promotion
  (
    code CodePromotionnel,
    noProduit NoProduit,
    début Date,
    fin Date,
    prix Montant_CAD,
    CONSTRAINT Promotion_cc0 PRIMARY KEY (noProduit, code),
    CONSTRAINT Promotion_ce0 FOREIGN KEY (noProduit) REFERENCES Produit,
    CONSTRAINT Promotion_Periode CHECK (début <= fin)
  );
```

Exemple 0
Procédure SQL

La procédure d'entretien mensuel consiste à

1. appliquer une variation uniforme du prix des produits.
2. supprimer les promotions périmées relativement à une date donnée.

Version ISO

```
CREATE OR REPLACE PROCEDURE Entretenir
  (taux NUMERIC(6,4), d DATE)
BEGIN ATOMIC
  UPDATE Produit
    SET prix = prix * taux ;
  DELETE FROM Promotion
    WHERE fin < d ;
END;
```


MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MiTiS — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Pourquoi ATOMIC ?
Pour préserver l'atomicité de la transaction.
Voir propiétées ACID.

Fonction - définition

Fonction – Syntaxe simplifiée (PostgreSQL)

function_def ::=
CREATE [OR REPLACE] FUNCTION *name* ([*argument* [, ...]])
RETURNS *ret_type*
{ *other_function_qual* } ...
funtion_body

argument ::=
[*arg_mode*] [*arg_name*] *arg_type* [DEFAULT *default_expr*]

arg_mode ::=
IN | VARIADIC

ret_type ::=
type_denotation
| TABLE ([*column_name column_type* [, ...]])

2025-02-11 MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018 - 2025, MiTiS — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

other_fun_qual ::=
WINDOW
| TRANSFORM { FOR TYPE ***type_name*** } [, ...]
| IMMUTABLE | STABLE | VOLATILE | [NOT] LEAKPROOF
| CALLED ON NULL INPUT | RETURNS NULL ON NULL INPUT | STRICT
| [EXTERNAL] SECURITY INVOKER | [EXTERNAL] SECURITY DEFINER
| COST ***execution_cost***
| ROWS ***result_rows***
| SET ***configuration_parameter*** { TO ***value*** | = ***value*** | FROM CURRENT }
| AS '***obj_file***', '***link_symbol***'

arg_mode

En cas d'omission, le mode est IN.

Une fonction ne devrait utiliser que des arguments IN ou VARIADIC;

les modes OUT et INOUT ne sont présents que par souci de rétro-compatibilité avec les versions antérieures à 8.4.

ret_type

Une troisième option, non conforme au standard ISO, est possible

SETOF composite_type

le composite_type est celui des tuples d’une table préalable crée (CREATE TABLE) ou celui définit par un CREATE TYPE.

DEFAULT

En PostgreSQL, peut être remplacé par le symbole **=**.

Fonction – Syntaxe simplifiée (PostgreSQL) [suite]

function_body ::=
  RETURN *expression*
 | BEGIN ATOMIC [*statement* ;] ... END
 | LANGUAGE *lang_name* AS *string*
 | AS '***obj_file***', '***link_symbol***'

Fonction – Syntaxe simplifiée (PostgreSQL) [suite]

other_function_qual ::=
WINDOW
| TRANSFORM { FOR TYPE ***type_name*** } [, ...]
| IMMUTABLE | STABLE | VOLATILE
| *[NOT] LEAKPROOF*
| *CALLED ON NULL INPUT*
| *RETURNS NULL ON NULL INPUT*
| *STRICT*
| *[EXTERNAL] SECURITY INVOKER*
| *[EXTERNAL] SECURITY DEFINER*
| *COST* ***execution_cost***
| *ROWS* ***result_rows***
| *SET* ***configuration_parameter***
{ TO ***value*** *| =* ***value*** *| FROM CURRENT }*

2025-02-11 MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018 - 2025, MVSq — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

TRANSFORM { FOR TYPE ***type_name*** } [, ...] }
Lists which transforms a call to the function should apply. Transforms convert between SQL types and language-specific data types; see **CREATE TRANSFORM**. Procedural language implementations usually have hardcoded knowledge of the built-in types, so those don't need to be listed here. If a procedural language implementation does not know how to handle a type and no transform is supplied, it will fall back to a default behavior for converting data types, but this depends on the implementation.
WINDOW indicates that the function is a *window function* rather than a plain function. This is currently only useful for functions written in C. The WINDOW attribute cannot be changed when replacing an existing function definition.

IMMUTABLE
STABLE
VOLATILE
These attributes inform the query optimizer about the behavior of the function. At most one choice can be specified. If none of these appear, VOLATILE is the default assumption.
* IMMUTABLE indicates that the function cannot modify the database and always returns the same result when given the same argument values; that is, it does not do database lookups or otherwise use information not directly present in its argument list. If this option is given, any call of the function with all-constant arguments can be immediately replaced with the function value.
* STABLE indicates that the function cannot modify the database, and that within a single table scan it will consistently return the same result for the same argument values, but that

its result could change across SQL statements. This is the appropriate selection for functions whose results depend on database lookups, parameter variables (such as the current time zone), etc. (It is inappropriate for AFTER triggers that wish to query rows modified by the current command.) Also note that the current_timestamp family of functions qualify as stable, since their values do not change within a transaction.
* VOLATILE indicates that the function value can change even within a single table scan, so no optimizations can be made. Relatively few database functions are volatile in this sense; some examples are random(), currval(), timeofday(). But note that any function that has side-effects must be classified volatile, even if its result is quite predictable, to prevent calls from being optimized away; an example is setval().
For additional details see **Section 38.7**.

LEAKPROOF
LEAKPROOF indicates that the function has no side effects. It reveals no information about its arguments other than by its return value. For example, a function which throws an error message for some argument values but not others, or which includes the argument values in any error message, is not leakproof. This affects how the system executes queries against views created with the security_barrier option or tables with row level security enabled. The system will enforce conditions from security policies and security barrier views before any user-supplied conditions from the query itself that contain non-leakproof functions, in order to prevent the inadvertent exposure of data. Functions and operators marked as leakproof are assumed to be trustworthy, and may be executed before conditions from security policies and security barrier views. In addition, functions which do not take arguments or which are not passed any arguments from the security barrier view or table do not have to be marked as leakproof to be executed before security conditions. See **CREATE VIEW** and **Section 41.5**. This option can only be set by the superuser.

Fonction – Appel (PostgreSQL)

- Comme un terme fonctionnel au sein d'une expression
- Exemples
  - (5.00 + Rabais('p001', '2020-10-31')) * 2
  - SELECT idProduit, Rabais(idProduit, '2019-10-31') AS rabais FROM Produit

MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MTSS — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Procédure - définition

MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MTIS — CC BY-NC-SA 4.0

Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

PROCÉDURE – Syntaxe simplifiée (PostgreSQL)

procedure_def ::=
CREATE [OR REPLACE] PROCEDURE *name* ([*argument* [, ...]])
{ *other_procedure_qual* } ...
procedure_body

argument ::=
[*arg_mode*] [*arg_name*] *arg_type* [DEFAULT *default_expr*]

arg_mode ::=
IN | VARIADIC | OUT

2025-02-11 MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MiTiS — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

other_proc_qual ::=
TRANSFORM { FOR TYPE ***type_name*** } [, ...]
| [EXTERNAL] SECURITY INVOKER | [EXTERNAL] SECURITY DEFINER
| COST ***execution_cost***
| ROWS ***result_rows***
| SET ***configuration_parameter*** { TO ***value*** | = ***value*** | FROM CURRENT }
| AS '***obj_file***', '***link_symbol***'

arg_mode

En cas d'omission, le mode est IN.

Le mode OUT n'est disponible que pour certains langages dont plpgsql, PL/I, Pascal, Modula-2;

il n'est pas disponible pour les langages SQL, C, C++, Java et python.

Procédure – Syntaxe simplifiée (PostgreSQL) [suite]

procedure_body ::=
  BEGIN ATOMIC [*statement* ;] ... END
 | LANGUAGE *lang_name* AS *string*
 | AS '***obj_file***', '***link_symbol***'

Procédure – Syntaxe simplifiée (PostgreSQL) [suite]

other_procedure_qual ::=

TRANSFORM { FOR TYPE ***type_name*** } [, ...]

| *[EXTERNAL] SECURITY INVOKER* |
| *[EXTERNAL] SECURITY DEFINER*

| *COST* ***execution_cost***

| *ROWS* ***result_rows***

| *SET* ***configuration_parameter***
{ TO ***value*** | = ***value*** | *FROM CURRENT }*

MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MTSS — CC BY-NC-SA 4.0

Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Procédure – Appel (PostgreSQL)

- Grâce à l'instruction CALL
- Exemple
  - CALL Entretenir (1.10, '2020-10-24')

Les modalités de l'abstraction

- Les formes générales
- L'usage en SQL

MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MiTiS — CC BY-NC-SA 4.0

Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Les formes de l'abstraction

- lexicale
  - pré-traitement par substitution de textes ou de lexèmes
    - micro : textes
    - macro : lexèmes
- syntaxique
  - pré-traitement par substitution de contructions lexicalement et syntaxiquement intègres
- sémantique
  - création d'une entité « autonome » sémantiquement intègre
  - dont la mise en oeuvre est
    - interne (native, ...) : exprimée in situ dans le langage lui-même
    - externe (étrangère, ...) : importée depuis une source déterminée

MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MTIS — CC BY-NC-SA 4.0
Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

L'usage de ces formes en SQL

- SQL ISO ne propose que la forme sémantique
- Plusieurs dialectes SQL proposent la forme syntaxique en sus (ou en remplacement)

- Les formes non sémantiques doivent être utilisées avec parcimonie
  - uniquement quand le traitement désiré ne peut être obtenu adéquatement grâce à la forme sémantique

 © 2018-2025, MiTiS — CC BY-NC-SA 4.0

Le cas de PostgreSQL

- Il est recommandé de n'utiliser la forme synaxique que dans les cas suivants :
  - l'expression des automatismes
  - le traitement des exceptions
  - l'utilisation d'une langage autre que SQL pour spécifier le traitement

MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MiTiS — CC BY-NC-SA 4.0

Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Fonction – Syntaxe simplifiée (PostgreSQL) [suite]

function_body ::=

RETURN expression – sémantique interne scalaire

| BEGIN ATOMIC [statement ;] ... END – sémantique interne non scalaire

| AS '**obj_file**', '**link_symbol**' – sémantique externe

| LANGUAGE *lang_name* AS *string* – syntaxique

MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MTIS — CC BY-NC-SA 4.0

Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Exemple 0
Fonction SQL

Déterminer l'indicatif régional d'un numéro de téléphone canadien ou états-unien.

Version «macro» PostgreSQL

```
CREATE FUNCTION IndicatifRegional
  (t Telephone_CA)
  RETURNS VARCHAR(3)
LANGUAGE SQL AS
  $$
    SELECT SUBSTR(t, 2, 3) ;
  $$
```

Procédure – Syntaxe simplifiée du corps en PostgreSQL

procedure_body ::=

BEGIN ATOMIC [*statement* ;] ... END – sémantique interne

| AS '***obj_file***', '***link_symbol***' – sémantique externe

| LANGUAGE *lang_name* AS *string* – syntaxique

Exemple 0
Procédure SQL

La procédure d'entretien mensuel consiste à

1. appliquer une variation uniforme du prix des produits.
2. supprimer les promotions périmées relativement à une date donnée.

Version «macro» PostgreSQL

```
CREATE OR REPLACE PROCEDURE Entretenir
  (taux NUMERIC(6,4), d DATE)
LANGUAGE SQL AS
  $$
    UPDATE Produit
        SET prix = prix * taux ;
    DELETE FROM Promotion
        WHERE fin < d ;
  $$;
```

Synthèse

CORPS de ROUTINE
Syntaxe (PostgreSQL)

- La routine, aussi appelée *permantly stored module* (PSM), est un objet stocké dans la base de données, comme les domaines, les types, les tables et les vues.

- Le corps (***body***) d'une routine est la description textuelle du traitement dans un langage (***lang_name***) spécifié.

- Les langages utilisables
  - varient d'une dialecte à un autre ;
  - comprennent minimalement SQL ;
  - comprennent souvent Pascal, C, C++, Java et Python ;
  - comprennent souvent un « *PSM language* ».

- Chaque dialecte SQL semble avoir défini son propre *PSM language* plutôt que d'adhérer à celui prescrit par la norme ISO :
  - plpgsql est celui de PostgreSQL ;
  - PL/SQL est celui d'Oracle.

PL/pgSQL

- Le langage PL/pgSQL est décrit dans
  - https://docs.postgresql.fr/17/plpgsql.html
  - https://www.postgresql.org/docs/current/plpgsql.html

Rétro-compatibilité
Prepared statement – « The Very Olde Style » #3 (antérieurement à 8.0)

Avant les procédures et fonctions

L'encapsulation des commandes

```
PREPARE name [ ( data_type [, ...] ) ] AS statement
EXECUTE name [ ( parameter [, ...] ) ]
DEALLOCATE name
```

voir
https://www.postgresql.org/docs/current/sql-prepare.html

MCED_SQL_06a : Procédures et fonctions (v1226) — Christina Khnaisser et Luc Lavoie © 2018-2025, MiTiS — CC BY-NC-SA 4.0

Département d'informatique, Faculté des sciences, Université de Sherbrooke, Québec

Références

- [Loney2008]
  Loney, Kevin ;
  Oracle Database 11g: The Complete Reference.
  Oracle Press/McGraw-Hill/Osborne, 2008.
  ISBN 978-0071598750.
- [Date2012]
  Date, Chris J. ;
  SQL and Relational Theory: How to Write Accurate SQL Code.
  2nd edition, O'Reilly, 2012.
  ISBN 978-1-449-31640-2.
- Le site d'Oracle (en anglais)
  - http://docs.oracle.com/cd/E11882_01/index.htm
- Le site de PostgreSQL (en français)
  - http://docs.postgresqlfr.org